



University of
Salford
MANCHESTER

MSc Data Science Dissertation

**A Temporal Analysis of the Performance
Stability of Fraud Detection Models Under
Concept Drift**

@00805339

Supervisor: Dr. Mansooreh Yazdankhah

April – 2026

Abstract

Fraud detection algorithms are commonly used in modern financial systems to spot fraudulent transactions and reduce financial loss. Nevertheless, these models are usually assessed using static approaches that do not account for the dynamic structure of transaction data, where fraud trends and user behavior change over time. When models are used in real-world situations, this restriction may cause performance reduction.

This study assesses the effectiveness of retraining techniques in sustaining performance over time and looks into the temporal stability of fraud detection models under concept drift. A temporal evaluation framework was created using the IEEE-CIS fraud detection dataset. To replicate realistic deployment conditions, transactions were arranged chronologically and divided into multiple times frames. XGBoost, Random Forest, and Logistic Regression were used in a few experiments, such as retraining experiments, temporal model evaluation, and exploratory data analysis (EDA).

The findings provide clear evidence of concept drift by showing that model performance continually declines when applied to future data. Random Forest demonstrated more steady behavior, while XGBoost demonstrated the highest predictive performance but the largest risk to temporal variations. Logistic Regression served as a baseline with consistently low performance. Retraining strategies significantly improved performance, with fixed retraining at shorter intervals achieving the best results, and performance-triggered retraining offering a more efficient balance between performance and computational cost.

Overall, this study highlights the limitations of static evaluation and emphasizes the importance of temporal evaluation and adaptive retraining in maintaining robust fraud detection systems.

Contents

Abstract	2
List of Figures.....	6
List of Tables	7
List of Abbreviations	8
Chapter 1: Introduction	9
1.1 Introduction	9
1.2 Background.....	10
1.3 Problem Statement.....	11
1.4 Research Objective and Research Questions.....	12
1.5 Significance of the study	12
1.6 Thesis Statement	14
Chapter 2: Literature Review.....	14
2.1 Evolution of Financial Fraud Detection.....	14
2.2 Evolution of machine learning approaches in Fraud Detection	15
2.3 Class Imbalance in Fraud Detection	17
2.4 Temporal Nature of Fraud Data	19
2.5 Concept Drift in Machine Learning.....	20
2.6 Concept Drift in Fraud Detection.....	21
2.7 Retraining Strategy.....	22
2.8 Research Gap	23
Chapter 3: Methodology.....	24
3.1 Introduction	24
3.2 Research Approach.....	25
3.3 Dataset Considerations.....	25
3.4 Temporal Evaluation Framework.....	26
3.5 Training Data Strategy	26
3.6 Concept Drift Representation	27
3.7 Model Selection Rationale	27
3.8 Retraining Strategy Design.....	28
3.9 Evaluation Metrics Strategy	29
Chapter 4: Data Understanding and Exploratory Data Analysis.....	30
4.1 Data Description.....	30
4.2 Exploratory Data Analysis	32
4.2.1 Introduction.....	32
4.2.2 Class Imbalance Analysis	32
4.2.3 Temporal Ordering and Realistic Data Representation.....	34

4.2.4 Transaction Volume Across Time Windows	35
4.2.5 Fraud Rate Variation and Behavioral Shifts	36
4.2.6 Drift Quantification using CSI and PSI.....	37
4.2.7 Integrated Evidence of Concept Drift	39
4.2.8 Implications for Model Development	40
4.2.9 Summary of Findings.....	40
Chapter 5: Implementation	41
5.1 Introduction	41
5.2 Temporal Data Construction.....	42
5.3 Training Data Reduction and Class Imbalance Handling	42
5.3.1 Sampling Strategy Explanation	43
5.3.2 Reasoning Behind the Design	44
5.3.3 Consistency with Research Objective	44
5.4 Preprocessing Pipeline	45
5.5 Model Implementation.....	45
5.5.1 Logistic Regression.....	46
5.5.2 Random Forest	46
5.5.3 XGBoost	47
5.5.4 Unified Model Pipeline	47
5.5.5 Temporal Hyperparameter Tuning	48
5.5.6 Implementation Rationale.....	49
5.6 Temporal Evaluation Logic	49
5.7 Retraining strategy Implementation	50
5.7.1 No-Retraining Baseline	50
5.7.2 Fixed Retraining	50
5.7.3 Performance – Triggered Retraining.....	51
5.8 Performance Measurement	51
5.9 Summary	52
Chapter 6: Results and Discussions.....	52
6.1 Introduction	52
6.2 Overall Model Performance	52
6.3 Performance Degradation Over Time.....	53
6.4 Effect of Retraining Strategies.....	55
6.5 Analysis of Retraining Behaviour	56
6.5.1 Fixed retraining Behaviour	57
6.5.2 Performance – Triggered Retraining Behaviour.....	58
6.5.3 Identification of Best Strategy.....	59

Chapter 7: Limitations and Conclusions	60
7.1 Limitations of the study	60
7.2 Conclusions	61
References	62

List of Figures

Figure 3. 1: Temporal Windowing Strategy	26
Figure 4. 1: Distribution of fraud (1) and non-fraud (0) transactions.....	33
Figure 4. 2: Sample of chronologically ordered transactions	34
Figure 4.3: Transaction volume across time windows.....	35
Figure 4.4: Median transaction amount for fraudulent and non-fraudulent transactions across time windows	36
Figure 4.5: Median transaction amount across time windows	36
Figure 4.6: Top features ranked by PSI (early vs late comparison)	38
Figure 4.7: Top features ranked by CSI	39
Figure 6.1: PR-AUC vs Time Windows – No retraining for all models.....	53
Figure 6.2: Model performance over time under fixed retraining every 1 window (PR-AUC and ROC-AUC).....	57
Figure 6.3: Model performance over time under fixed retraining every 2 windows (PR-AUC and ROC-AUC).....	57
Figure 6.4: Model performance over time under fixed retraining every 3 windows (PR-AUC and ROC-AUC).....	58
Figure 6.5: Performance of models under performance-triggered retraining (PR-AUC and ROC-AUC over time)	59

List of Tables

Table 4. 1: Feature Descriptions	31
Table 5.1: Hyperparamter grid for models	48
Table 6.1: Average PR-AUC across models	52
Table 6.2: PR-AUC trend (no retraining).....	54
Table 6.3: Performance Degradation Summary	54
Table 6.4: PR-AUC comparison across retraining strategies for all models	55
Table 6.5: Adaptive retraining event	59

List of Abbreviations

PR-AUC	Precision-Recall Area Under Curve
ROC-AUC	Receiver Operating Characteristic Area Under Curve
SMOTE	Synthetic Minority Oversampling Technique
EDA	Exploratory Data Analysis
ML	Machine Learning
PSI	Population Stability Index
CSI	Characteristics Stability Index

Chapter 1: Introduction

1.1 Introduction

The rapid evolution of financial services in the digital environment has significantly transformed the global economic landscape, offering faster, more convenient, and highly interconnected transaction ecosystems. The increased use of online banking systems, mobile payment systems, and electronic commerce technologies has led to a substantial growth in financial transactions in the digital environment (Ozili, 2018; Vives, 2019). Although these technologies have enhanced the efficiency of financial institutions, they have also introduced new vulnerabilities that can be exploited for fraudulent activities (Arner et al., 2016). Consequently, financial fraud has become a major challenge in the modern financial environment, causing financial losses, damage to financial reputation, and undermining consumer trust in financial systems in the digital environment (Dal Pozzolo et al., 2018).

In the real world, fraud detection models are usually trained on previous datasets and are deployed for the purpose of continuously monitoring the flow of transactions (Dal Pozzolo et al., 2018). The performance of the model is expected to remain consistent even under the influence of changing customer behavior, market trends, and fraudster strategies. The financial transaction environment is usually dynamic by nature, as the data distribution is constantly subject to change. This can be attributed to various factors, including seasonal spending patterns, technological developments, and the strategies employed by fraudsters (Baesens et al., 2021).

The motivation for this study is based on the dynamic nature of financial transaction environments, where behavior patterns constantly change over time. For instance, fraudsters often adjust their methods to circumvent detection systems, and customers' behaviors also tend to alter in response to socio-economic factors, technological advancements, and regulatory shifts (Ngai et al., 2011). As a result, there is a potential for a degradation in the performance of models that relied on training data from the past. Therefore, analyzing the temporal performance of machine learning based fraud detection models has become an important research problem (Gama et al., 2014; Lu et al., 2018). This research is centered on investigating the temporal stability of models used for detecting fraud in the presence of concept drift, with the ultimate aim of providing insights that support the development of more robust and adaptive fraud detection systems.

1.2 Background

Financial institutions today utilize automated fraud detection systems to process large volumes of transactional data and detect suspicious activities in real time (Dal Pozzolo et al., 2018). Traditionally, rule-based systems were commonly used for fraud detection. However, although these systems offered initial effectiveness in detecting fraud, their inflexibility to accommodate new fraud schemes became a key limitation (Phua et al., 2010). As financial transactional networks grew in scale and size, so did the need to address the limitations of rule-based systems.

Thus, machine learning-based fraud detection systems have been proposed to address the aforementioned limitations (Bhattacharyya et al., 2011). The proposed systems use historical information related to the transactions to learn discriminative features that can differentiate between legitimate and fraudulent transactions. In this regard, various machine learning-based methods, including Logistic Regression, Decision Trees, Ensemble methods, and Boosting, have been successfully employed for fraud detection purposes (Dal Pozzolo et al., 2018; Baesens et al., 2021). This can help the financial institutions handle the high volume of transactions efficiently.

However, the financial environment is constantly changing and this impacts on the distribution of the data. Customer behavior, for instance, may constantly change, depending on factors such as seasonality, economic conditions, and technological factors. Similarly, fraudsters also adapt their strategies in order to take advantage of any new weaknesses that they discover (Dal Pozzolo et al., 2015). This, in essence, creates a non-stationary data environment, where the statistical properties of the data are constantly changing. This is commonly referred to as a concept drift (Gama et al., 2014).

Concept drift is a phenomenon that characterizes how statistical characteristics of data vary over time, which can have a big effect on how well machine learning models predict outcomes (Lu et al., 2018). In reality, a lot of fraud detection systems don't have strong methods for methodically accessing or adjusting to concept drift (Baesens et al., 2021). As a result, models may eventually become out of date, increasing operational risks and decreasing detection accuracy. This emphasizes the necessity for studies that focus on adaptive modelling techniques and temporal evaluation in fraud detection systems.

1.3 Problem Statement

Although machine learning techniques have demonstrated strong predictive capabilities in fraud detection tasks, their evaluation has often been conducted using static experimental methodologies that do not reflect real-world operational conditions (Dal Pozzolo et al., 2018). Most existing studies rely on conventional train-test split approaches, where models are trained and evaluated on datasets assumed to follow stable distributions. Such evaluation strategies fail to capture the temporal dynamics of financial transaction data and do not provide insights into how model performance changes over time.

Concept drift is a major challenge that affects the reliability of the output of a fraud detection system, since the patterns of fraud and the behaviors of customers may change over time, making the model to become outdated (Gama et al., 2014; Lu et al., 2018). This has an impact on the model's performance, which may lead to an increase in false positives, thus inconveniencing legitimate customers, and false negatives, which allow fraudulent activities to go undetected (Dal Pozzolo et al., 2018). Despite the significance of this problem, limited research has been done on the systematic analysis of the temporal stability of fraud detection models under concept drift.

In addition, there is a lack of understanding about relative effectiveness of different machine learning techniques in terms of their robustness and ability to adapt to evolving transaction environments (Baesens et al., 2021). The effectiveness of the process of retraining, sequential evaluation frameworks, and the mechanism of adaptive learning in maintaining the stability of performance has not been adequately investigated. This gap in existing literature creates uncertainty regarding the practical application of machine learning based fraud detection systems.

Therefore, this study seeks to address the need for a structured temporal evaluation framework that enables the systematic analysis of fraud detection model performance over time. By examining the effects of concept drift and evaluating adaptive modeling strategies, this study aims to contribute to the development of more reliable and resilient fraud detection solutions.

1.4 Research Objective and Research Questions

The main objective of the research is to explore the temporal stability of machine-learning based fraud detection model performance with the presence of concept drift. The research aims to bridge the gap between the theoretical evaluation of the models and the practical challenges associated with its deployment of financial fraud detection systems.

The specific objectives of this research are as follows:

- To construct a temporal evaluation framework that examines how fraud detection model performance evolves across sequential time periods.
- To investigate how concept drift influences the predictive effectiveness of machine learning models in dynamic transaction environments.
- To assess and contrast the stability and robustness of selected fraud detection algorithms under changing data conditions.
- To determine the extent to which retraining strategies contribute to maintaining model performance over time.
- To generate practical insights that support the design and deployment of adaptive fraud detection systems.

This research is guided by the following research questions:

- I. How does the predictive performance of fraud detection models change over time in the presence of concept drift?
- II. Which machine learning models demonstrate greater stability and robustness in evolving financial transaction environments?
- III. How can temporal evaluation frameworks provide more realistic insights into model effectiveness compared to static evaluation approaches?
- IV. To what extent do retraining strategies improve the long-term reliability of fraud detection systems?

1.5 Significance of the study

The study is significant from both practical and academic perspective, especially with regard to the application of research in the domain of financial cybersecurity and machine learning deployment in dynamic environments. The research is relevant from an academic perspective

as it contributes to the existing pool of knowledge with regard to domain of concept drift and temporal model evaluation, as it offers a framework for the evaluation of the performance stability of fraud detection systems. While the majority of the research has focused on improving the accuracy of the results obtained from machine learning algorithms, limited attention has been given to the evaluation of the stability of the performance of the models in a real-world environment, as this research offers a methodological extension to the domain of adaptive machine learning for the detection of fraud.

In addition, the research helps to advance the theory of non-stationary data environments, which is of increasing interest for wide range of applications outside the domain of financial fraud detection, including cybersecurity monitoring, intrusion detection, and behavioral analytics. The temporal evaluation framework developed in the research can be used as a systematic approach to test the robustness of models under non-stationary data environments, which can help to advance the research discourse on the stability of model performance, model lifecycle management, and adaptive learning strategies in machine learning models.

From an industry perspective, financial institutions depend on effective and efficient fraud detection systems to reduce financial loss and ensure regulatory compliance and customer trust. To keep up with the rate and volume of digital transactions, these institutions need to ensure that fraud detection systems can operate for longer deployment times. By using knowledge of temporal aspects to make effective decisions on choosing and retraining predictive models, these institutions can use the research study's findings to support the development of fraud detection systems that are better adapted to dynamic environments.

Furthermore, this research offers useful insights into the compromises that must be made in relation to prediction accuracy, computation efficiency, and model stability. This is particularly useful for practitioners who are tasked with deploying machine learning-based solutions for detecting financial fraud. By including temporal performance analysis and concept drift analysis within this research, there is an attempt to bridge the gap that exists between theoretical research in machine learning and real-world financial system's needs. Ultimately, the outcome of this research is expected to be useful for developing financial systems that are secure and efficient for detecting financial fraud.

1.6 Thesis Statement

This thesis argues that traditional static evaluation approaches are insufficient for accessing the real-world effectiveness of machine learning-based fraud detection models due to the presence of concept drift in financial transaction data. By adopting a temporal evaluation framework and analyzing the performance stability of predictive models over sequential time periods, this research demonstrates the importance of adaptive modeling strategies in developing reliable and resilient fraud detection systems for dynamic financial environments.

Chapter 2: Literature Review

2.1 Evolution of Financial Fraud Detection

Financial fraud detection techniques have undergone a tremendous transformation over the years, mainly because of the growth of financial systems and the complexity of transactional environments. In early days, financial fraud detection processes relied primarily on rule-based identification of transactions, where predefined threshold reflected experts' understanding of the relationship between logical rules and security mechanisms. The simplicity of this technique made it easier for implementation and understanding, but it had a limitation of being inflexible, as it had to be manually updated. However, as fraud techniques became more complex, it became difficult for simple rule-based systems to adapt, thereby decreasing their detection capability.

With the growth of online banking, mobile payments, and e-commerce platforms, the volume and diversity of financial transactions increased substantially. This led to the adoption of machine learning techniques, which enabled systems to automatically learn patterns from historical data and detect anomalies more effectively. Due to their capacity to handle large-scale data and increase detection accuracy supervised learning algorithms like Random Forest, Decision Trees, and Logistic Regression became widely adopted. When compared to conventional rule-based systems, machine learning techniques have been demonstrated to provide improved scalability and predictive performance (Chen et al., 2025).

Recent advancements in fraud detection have been focused on sophisticated techniques, including ensemble learning, deep learning, and intelligent systems (Compagnino et al., 2025).

These techniques are expected to detect complex, non-linear relationships with transactions. They can increase the accuracy of detection, particularly in dynamic conditions, intelligent systems that can adjust to changing patterns are becoming more and more popular, particularly for real-time decision making (Chen et al., 2025).

However, the majority of current methods still depend on the presumption that data distributions are steady over time, regardless of these developments. In reality, user behavior and fraud tactics are always changing, making financial transaction data extremely dynamic. Because of this, models that perform well during initial training may perform worse when applied to subsequent data.

Therefore, rather than depending just on static performance measurements, it is crucial to assess how these models function over time in the presence of changing data distributions, even though fraud detection approaches have greatly improved in terms of accuracy and complexity.

2.2 Evolution of machine learning approaches in Fraud Detection

As financial systems have grown more complicated, the development of fraud detection strategies has clearly evolved from traditional statistical methods toward more advanced machine learning techniques. Rule-based systems and simple statistical models were the backbone of early fraud detection techniques, which used predefined criteria to spot suspicious transactions. These systems were appropriate for early financial settings because they were simple to put together and offered a high level of interpretability. However, because they were unable to adapt to new or unidentified fraud patterns, their effectiveness was hindered. According to Phua et al. (2010), rule-based systems were ineffective in dynamic contexts because they required constant manual modifications.

Acknowledging these constraints, scientists started investigating data-driven strategies utilizing machine learning and statistical methods. Because of its simplicity and ability to model relationships between variables, logistic regression became one of the earliest machine learning models applied to fraud detection. Logistic regression has been shown by Bahnsen et al. (2016) to be a useful model for fraud detection, especially in situations where costs are a concern. Despite its benefits, logistic regression's ability to capture complex fraud patterns is constrained by its assumption of a linear relationship between input characteristics and the target variable.

More adaptable models, including decision trees, were developed to overcome this restriction. Compared to traditional statistical models, decision trees provided better interpretability and nonlinear decision boundaries. However, it was discovered that single decision trees were extremely sensitive to noise and prone to overfitting, which decreased their reliability in practical applications.

As a result, ensemble learning techniques were created to enhance model stability and performance. In order to overcome the drawbacks of single decision trees, Breiman (2001) developed the random forest, which combines several trees to lower variance and enhance generalization. Because random forests can handle high-dimensional data and noisy characteristics, studies have demonstrated that they perform well in fraud detection tasks. In contrast to more advanced models, random forests may still have trouble identifying extremely complex patterns in data despite these benefits.

Gradient boosting techniques were employed to further improve predicting performance. XGBoost, which was introduced by Chen and Guestrin (2016), builds models one after the other by fixing mistakes from earlier versions. In numerous fraud detection applications, this method has been demonstrated to deliver state-of-the-art performance. Because of their great accuracy, boosting models are therefore frequently used in practice.

Recent research, however, indicates that boosting models may become less stable as a result of their increased complexity. These models can be extremely sensitive to shifts in the distribution of the data, even though they work well in static situations. Fraud detection systems operate in non-stationary environments where patterns change over time, according to Dal Pozzolo et al. (2018). In certain situations, models that were trained on historical data may perform worse when applied to future data.

In response to these challenges, researchers have looked into more advanced solutions to similar problems, such as deep learning models like neural networks and LSTM architecture. According to Fiore et al. (2019), deep learning models can enhance detection performance by capturing intricate patterns in transactional data. However, the practical usefulness of these models is limited due to their lack of interpretability, computational cost, and frequent requirement for significant volumes of data.

A recurrent problem persists despite the ongoing development of machine learning approaches for fraud detection. The majority of research focuses on increasing predicted accuracy and

evaluates models using static datasets. Real-world situations, where data distributions fluctuate over time and model performance may deteriorate, are not represented by these methods.

As a result, even though fraud detection accuracy has significantly improved, little is known about how various machine learning models respond to temporal shifts in data distribution. This emphasizes the necessity for studies that look at both model performance and performance stability over time, especially when concept drift is present.

2.3 Class Imbalance in Fraud Detection

One of the biggest obstacles to financial fraud detection is class imbalance, since fraudulent transactions usually make up only a very small proportion of the overall dataset. Fraud rates in real-world financial systems often fall below 1%, which leads to severely skewed class distributions. This imbalance can significantly affect the performance of machine learning models, as most algorithms are designed to optimize overall accuracy and therefore tend to favor the majority class (Dal Pozzolo et al., 2015). As a result, models may achieve high overall accuracy while failing to correctly identify fraudulent cases.

Early research in fraud detection identified that traditional evaluation metrics such as accuracy are not suitable for imbalanced datasets, as a model can achieve high accuracy by simply predicting all transactions as non-fraudulent. As a result, alternative evaluation metrics such as precision, recall, and Area Under the Precision-Recall Curve (PR-AUC) have been widely adopted to provide a more realistic assessment of model performance (Davis & Goadrich, 2006).

Numerous data-level and algorithm-level strategies have been put forth to address the problem of class imbalance. Oversampling, which replicates instances of minority classes to balance the dataset, is one of the most popular methods. In order to enhance model performance, Chawla et al. (2002) developed the Synthetic Minority Over-sampling Technique (SMOTE), which creates synthetic minority class samples. Although SMOTE has been demonstrated to be successful in increasing recall, it may introduce noise and does not always accurately represent fraud trends.

Undersampling, in which a selected portion of instances from the majority class is used to balance the dataset, is another often employed technique. By limiting the influence of the majority class within the dataset, controlled undersampling can lead to improved fraud

detection performance, as reported by Dal Pozzolo et al. (2015). However, this strategy can cause the majority class to lose important information, which could have an impact on the generalization of the model.

Cost-sensitive learning, which imposes harsher penalties for misclassifying fraudulent transactions, has been proposed as an alternative to data-level methods for addressing class imbalance. By giving the minority class priority, cost-sensitive models can enhance detection performance, as demonstrated by Bahnsen et al. (2016). Nevertheless, these methods may not generalize well across various datasets and require careful cost parameter tweaking.

Despite the effectiveness of these techniques, a critical limitation exists in how they are applied in most studies. The majority of existing research treats class imbalance as a static problem, applying sampling techniques on randomly shuffled datasets. This approach ignores the temporal nature of financial transaction data and may lead to data leakage, where information from future transactions is inadvertently used during model training. Such leakage can result in overly optimistic performance estimates that do not reflect real-world deployment conditions.

Real-world fraud detection systems receive data in a sequential manner across time, and models are trained on past data before being used for transactions in the future. As a result, sampling strategies need to be used carefully to maintain the data's temporal integrity. The majority of research focuses on increasing accuracy over preserving realistic evaluation settings, hence this aspect has gotten little attention in the literature.

More recent methods have started investigating time-aware sampling techniques, which incorporate class imbalance handling within a temporal framework, in order to overcome this constraint. These methods highlight the need to maintain the chronological order of transactions while only using sampling to training data. This guarantees that models do not have access to future data while being trained, leading to a more accurate assessment of performance.

Building on this viewpoint, the current work uses a controlled sampling approach that is consistent with deployment scenarios in the real world. Class imbalance is only addressed inside the training data using a controlled negative-to-positive ratio, as opposed to using sampling globally throughout the dataset. This method preserves the natural distribution of data during testing while guaranteeing that the model is exposed to a balanced representation of classes during training.

Unlike traditional methods that treat class imbalance independently of time, this approach integrates class imbalance handling within a temporal evaluation framework, ensuring that the chronological structure of the data is preserved. This not only improves the realism of model evaluation but also provides a more accurate representation of how models perform in dynamic environments.

Therefore, while existing studies have made significant contributions in addressing class imbalance, they largely overlook the importance of temporal consistency. By incorporating time-aware sampling strategies, this study provides a more realistic and practical approach to handling class imbalance in fraud detection systems.

2.4 Temporal Nature of Fraud Data

Financial transaction data is inherently temporal, as transactions occur sequentially over time rather than as independent observations. Each transaction is influenced by past behavior, contextual patterns, and external factors, making time an important dimension in fraud detection. Unlike many traditional machine learning problems where data can be randomly shuffled without affecting model performance, fraud detection data contains temporal dependencies that must be preserved for meaningful analysis.

Early fraud detection studies often treated datasets as static and applied random train-test splits to evaluate model performance (Ngai et al., 2011). While this approach simplifies model development, it fails to reflect real-world deployment conditions. In practice, fraud detection systems are trained on historical data and applied to future transactions, where patterns may differ significantly. Random splitting can therefore lead to unrealistic performance estimates, as models may unintentionally learn from future data.

Dal Pozzolo et al. (2018) emphasized that there is a strong temporal ordering in financial transaction streams, implying that legitimate user behavior and fraudulent patterns vary over time. For example, customers' expenditure patterns can vary due to changes in lifestyle, seasonal effects, or economic conditions. At the same time, fraudsters are constantly looking for new ways to evade detection systems by introducing new patterns that were not present in the past.

Because of these temporal shifts, fraud detection is no longer a static classification task but rather a dynamic problem. If models trained on historical data are unable to adjust to changing

patterns, they may become less useful when applied to new data. This problem is strongly linked to concept drift, where model performance can be negatively impacted by changes in data distribution over time (Gama et al., 2014).

To address this, time-aware approaches such as time-based splitting and sliding window techniques have been proposed. These methods preserve the chronological order of data and allow models to be evaluated on future time periods, providing a more realistic assessment of performance (Jurgovsky et al., 2018). However, such approaches are still not consistently applied in many fraud detection studies.

Therefore, the temporal nature of fraud data plays a critical role in model evaluation and performance. Ignoring time can lead to misleading results, while incorporating temporal structure enables a more accurate understanding of how models behave in real-world, evolving environments.

2.5 Concept Drift in Machine Learning

The performance of machine learning models can be greatly impacted by concept drift, which is defined as changes in the underlying data distribution over time. It is frequently assumed that the relationship between the target variable and the input information stays constant in normal machine learning settings. However, this assumption is not true in many real-world applications, such as fraud detection (Gama et al., 2014).

Concept drift is the phenomenon wherein models trained on historical data become less accurate when applied to fresh data due to changes in the statistical features of data. These alterations may result from a number of things, including shifting user behavior, changes to the external environment, or adjustments to system procedures. Because of this, models that do well at first may eventually become less predictive.

In the literature, various forms of thought drift have been recognized. Gradual drift refers to progressive and continuous changes in the data distribution, whereas sudden drift occurs when there is an abrupt shift in the underlying data patterns. According to Widmer and Kubat (1996), recurring drift happens when previously observed patterns reappear over time, while incremental drift is characterized by minor changes that accumulate gradually. Maintaining model performance is a challenging task across all types of drift.

To address concept drift, various techniques have been proposed, including drift detection methods and adaptive learning approaches. Drift detection techniques, such as Adaptive Windowing (ADWIN) and Drift Detection Method (DDM), track changes in data distribution or model performance and trigger updates when significant changes are identified (Bifet & Gavalda, 2007). Online learning and other adaptive learning strategies constantly update models in response to new data.

However, the majority of current research concentrates on identifying instances of drift rather than comprehending how it impacts model performance over time. Although drift detection offers a way to spot changes, it doesn't completely explain how various models react to changing data. Because of this, there is a gap in the literature where detection is prioritized over performance analysis.

Consequently, it is crucial to comprehend how concept drift affects model performance, especially in dynamic settings where sustaining steady performance over time is crucial.

2.6 Concept Drift in Fraud Detection

Concept drift is particularly significant in fraud detection due to the adversarial nature of fraudulent activities. In contrast to many other applications, intelligent adversaries deliberately try to evade detection systems in fraud detection. Because of this, fraud patterns are always changing, making it challenging for static models to continue to be useful.

Fraud detection systems function in extremely dynamic contexts where both genuine user behavior and fraudulent techniques vary over time, as noted by Dal Pozzolo et al. (2018). For instance, fraudsters might create fresh attack strategies, take use of new technologies, or concentrate on other kinds of transactions. Models trained on previous data might not be able to detect the new patterns introduced by these changes.

Because of this, models that perform well during initial training could perform worse when used in real-world scenarios. Without constant observation, this deterioration, which is frequently slow, can go unrecognized. This may eventually result in lower detection rates and higher financial losses.

A number of strategies have been put out to deal with the concept drift in fraud detection. Online learning methods update models continuously as new data becomes available, allowing

them to adapt to changing patterns. According to Krawczyk et al. (2017), ensemble approaches enhance robustness by combining several models that have been trained over various times. Additionally, when notable changes are found, model updates can be initiated using drift detection techniques.

Despite these developments, many methods prioritize enhancing adaptability over examining performance stability. To put it another way, although models are made to adjust to changes, little is known about how their performance varies over time. This is especially crucial for fraud detection since even slight performance drops might have severe consequences.

Therefore, concept drift in fraud detection is not only a technical challenge but also a practical concern, highlighting the need to study how models degrade and adapt over time in dynamic environments.

2.7 Retraining Strategy

Retraining techniques are essential for preserving machine learning models' performance in dynamic settings where concept drift is an issue. Models trained on historical data may become out of date when data distributions change over time, which would result in a decline in performance (Gama et al., 2014). In order to maintain models' relevance and efficacy in identifying novel patterns, retraining procedures are therefore required.

Periodic retraining is one of the most popular methods, in which models are refreshed with fresh data at predetermined intervals. Because of its simplicity and convenience of use, this approach is commonly used. However, its effectiveness depends heavily on the chosen retraining frequency. While infrequent retraining may allow models to deviate dramatically from current data trends, frequent retraining may result in needless computational expense.

Another approach is online learning, where models are continuously updated as new data becomes available (Gama et al., 2010). This enables models to respond quickly to changes in data distribution. However, this learning method may be vulnerable to noise, which can result in unstable predictions, especially in environments where changes in data are not regular or predictable.

Drift-based retraining strategies aim to address these limitations by updating models only when significant changes in data distribution are detected. Techniques such as Adaptive Windowing

(ADWIN) and Drift Detection Method (DDM) are used to monitor performance and trigger retraining when necessary (Bifet & Gavalda, 2007; Gama et al., 2004). While this approach improves efficiency, it introduces additional complexity and requires careful tuning of detection parameters.

Although these strategies are available, there are still a number of restrictions. Many retraining techniques are complicated and might not work well in practical settings. Furthermore, the majority of research assesses retraining techniques based on improvements in overall accuracy, paying little consideration to how retraining impacts performance stability over time. Additionally, there are no standardized frameworks for evaluating various retraining approaches in actual, time-dependent scenarios.

Simpler retraining techniques are frequently chosen in real-world fraud detection systems because of their interpretability and convenience of implementation. Models can be assessed under various retraining settings to comprehend their behavior throughout time instead of depending on intricate adaptive procedures. For instance, models can be periodically retrained using newly available data to capture emerging trends, or they can be trained once and applied to future data without changes.

This viewpoint emphasizes how crucial it is to utilize retraining techniques and methodically assess how they affect model performance over time. The trade-off between model stability, flexibility, and computing efficiency can be evaluated by comparing models under various retraining situations within a time-aware evaluation framework.

Consequently, there is still a need for useful methods to assess how retraining affects model performance in dynamic contexts, even though current research has concentrated on creating sophisticated retraining strategies. By analyzing model behavior under various retraining scenarios, this study advances this field by shedding light on how well retraining techniques work when idea drift is present.

2.8 Research Gap

Previous research on fraud detection has mostly concentrated on utilizing machine learning models to increase predictive accuracy. However, when these strategies are applied in practical scenarios, a number of significant limitations still exist.

First, static datasets with randomly mixed train-test splits are used in the majority of research. This can produce unrealistic performance statistics because it overlooks the temporal aspect of transaction data.

Second, despite the widespread discussion of concept drift, current research primarily concentrates on identifying drift rather than examining how model performance deteriorates with time. There is minimal comparison of how various models respond to shifting data distributions.

Third, class imbalance methods like SMOTE and undersampling are frequently used without taking temporal order into account, which could cause data leakage and lower the accuracy of findings.

Lastly, retraining is often recommended as a solution, but little research has been done to compare effective retraining techniques or assess how they affect long-term performance stability.

Therefore, the key gap is the lack of a time-aware evaluation framework that jointly considers temporal data structure, concept drift, and retraining strategies.

This study addresses this gap by analyzing the temporal performance stability of fraud detection models under concept drift, while also evaluating the impact of retraining on model performance over time.

Chapter 3: Methodology

3.1 Introduction

This chapter presents the methodological framework used to examine the temporal stability of machine learning models for fraud detection under concept drift conditions. The study is designed as an experimental investigation that evaluates how predictive performance evolves over time in a dynamic financial transaction environment.

The methodology combines temporal windowing, controlled training strategies, multiple machine learning models, and adaptive retraining mechanisms to provide a comprehensive evaluation of model performance over time.

3.2 Research Approach

This study adopts an experimental and comparative research approach. The key aim is not to create a new model, but to analyze how models behave over time when exposed to concept drift.

Fraud detection is by nature a dynamic problem, where transaction patterns change continuously because of evolving user behavior and fraud tactics. Therefore, the static evaluation of models is not appropriate to capture the performance of models in real-life conditions. To overcome the static evaluation problem, this study uses a time-aware evaluation strategy that maintains the transactions chronological order.

The experimental setup involves training models on earlier segments of the dataset and then testing them on later segments. This enables the observation of performance degradation over time and compares the stability of different models under identical conditions. In addition, retraining strategies are incorporated to evaluate how adaptive approaches can improve model reliability.

3.3 Dataset Considerations

The IEEE-CIS Fraud Detection dataset is used as the foundation for this study due to its real-world relevance and temporal characteristics. The dataset contains anonymized e-commerce transactions with large number of features describing transaction behavior, device attributes, and user identity information.

While detailed data exploration is presented in Chapter 4, it is worth noting that the dataset contains a temporal feature (TransactionDT), which enables the ordering of transactions over time. This characteristic is essential for implementing a temporal evaluation framework, as it allows the study to stimulate how models operate in real-world situations.

In addition, the dataset exhibits significant class imbalance, where fraudulent transactions account for only a small portion of the overall data. This imbalance is taken into account during model evaluation and influences the choice of performance metrics used in the study.

3.4 Temporal Evaluation Framework

A temporal evaluation framework is used to assess performance over time. Instead of randomly splitting the data, the dataset is structured into sequential time-based segments that preserve natural order of transactions. This approach is significant as it ensures that the model is always evaluated on unseen data that was not available during training. This is similar to real-world applications, as models need to generalize to unseen and potentially different data distributions.

Moreover, temporal segmentation enables the study to track performance across multiple time periods, enabling identification of trends such as performance degradation and improvement. This framework is crucial in this research as it enables the assessment of the impact of concept drift.

Figure 3.1 illustrates the temporal windowing strategy used in this study. As shown in the figure, earlier windows are used to train the model, while later windows are used for sequential testing.

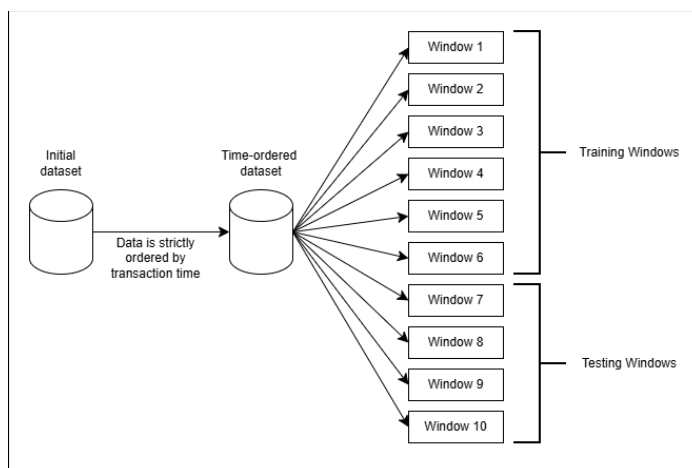


Figure 3. 1: Temporal Windowing Strategy

3.5 Training Data Strategy

Considering the huge volume of the data, as well as the issue of class imbalance, a controlled approach is used for the training process. The selection of the data for the training process is such that it covers all the cases of fraud, while at the same time, it remains computationally efficient.

In this approach, the majority of data does not overwhelm the models, allowing them to discover significant patterns. It is important to note that this procedure is only applied to the training data, whereas the test data remains unchanged.

3.6 Concept Drift Representation

The concept drift is also demonstrated in the study through the performance of the models at different time intervals. While the idea of concept drift is directly not computed using statistical techniques, the focus is more on the performance trends.

The decline in performance in later windows is considered indicative of the concept drift. This is more in line with the real-world scenario where drift is frequently detected by decreased model efficiency than direct measurements.

This study illustrates challenges in maintaining steady performance of fraud detection models and offers insights into how models react to changing settings by monitoring performance over several time periods.

3.7 Model Selection Rationale

The study evaluates three machine learning models: Logistic Regression, Random Forest, and Gradient Boosting (XGBoost), which represent different levels of complexity and performance in the context of fraud detection.

The first one, Logistic Regression, was chosen because it is the most simple, straightforward and popular method for detecting fraud. The linear decision boundary of the Logistic Regression model makes it extremely sensitive to changes in the data distribution. This is actually beneficial in this study because it enables a more direct analysis of how concept drift influences the overall performance of the model.

The second one, Random Forest, was chosen as a stability-focused approach. The ability of Random Forest to capture nonlinear relationships and be resilient to noise makes it suitable for this study. As an ensemble learning method, it is well suited for tabular fraud detection data and is a popular approach in industry applications. Its aggregated decision-making structure is

known to provide more stable performance over time compared to boosting-based methods, making it a good benchmark for assessing model stability.

The third one, Gradient Boosting (XGBoost), was chosen because of its high predictive accuracy and reputation as a cutting-edge technique in fraud detection as a high-performance model. However, despite its high performance, boosting based models are often more sensitive to changes in data distribution, which may result in reduced stability over time. This makes them particularly relevant for evaluating the impact of concept drift in dynamic fraud detection environments.

The inclusion of these models enables a structured comparison of baseline, stability-focused, and high-performance approaches, allowing the study to evaluate trade-offs between interpretability, predictive performance, and temporal stability under concept drift conditions.

3.8 Retraining Strategy Design

To examine whether model updating can improve long-term fraud detection performance under concept drift, this study evaluates multiple retraining strategies. The purpose of including retraining is to stimulate how fraud detection models may be maintained in real-world deployment settings, where transaction patterns evolve over time and previously trained models may gradually lose effectiveness.

The first strategy used in this study is a no-retraining baseline. Under this approach, each model is trained once using the initial historical transaction windows and then evaluated on all subsequent future windows without any further updates. This strategy serves as the reference condition for the study, as it allows performance degradation to be observed in the absence of adaptation. It provides a clear view of how model effectiveness changes when the underlying data distribution shifts over time.

The second strategy is fixed-interval retraining. In this approach, models are periodically retrained after a predefined number of temporal windows. Rather than assuming a single retraining schedule, this study considers multiple retraining frequencies in order to examine how update frequency influences model stability. More frequent retraining is expected to improve adaptation to recent transaction patterns, while less frequent retraining reflects a more resource-efficient but potentially less responsive deployment strategy. The inclusion of

different fixed retraining intervals allows the study to assess the trade-off between adaptability and operational cost.

Performance-triggered retraining is the third strategy. This method does not retrain the model on a regular basis, in contrast to fixed retraining. Rather, retraining is only done when the model's performance significantly deteriorates in comparison to a previous reference level. This approach is representative of a more flexible maintenance procedure, in which model updates are motivated by indication of diminished efficiency rather than by a set time. Because concept drift may not happen frequently and performance degradation may vary based on the model and the data segment being assessed, such an approach is very relevant in fraud detection.

Static deployment, scheduled updating and adaptive updating are the three methods of model maintenance represented by these retraining techniques. When combined, they give an organized framework for analyzing whether retraining enhances temporal performance stability which approach strikes the most sensible balance between retraining frequency and prediction reliability.

3.9 Evaluation Metrics Strategy

The evaluation framework in this study is designed to assess both predictive performance and temporal stability of fraud detection models under concept drift conditions. Given the highly imbalanced nature of fraud detection data, the choice of evaluation metrics is critical to ensure meaningful interpretation of results.

Precision-Recall Area Under Curve (PR- AUC) is used as a primary evaluation metric. Since fraudulent transactions represent a minority class, PR-AUC provides a more informative measure of model performance by focusing on the model's ability to correctly identify fraudulent instances. Unlike accuracy, which can be misleading in imbalanced datasets, PR-AUC captures the trade-off between precision and recall, making it more suitable for fraud detection tasks.

In addition to PR-AUC, Receiver Operating Characteristic Area Under Curve (ROC-AUC) is used to evaluate overall classification performance. ROC-AUC provides a general measure of the model's ability to distinguish between fraudulent and legitimate transactions across

different thresholds. The combined use of PR-AUC and ROC-AUC allows for a comprehensive assessment of model effectiveness.

To evaluate temporal stability, model performance is recorded across all sequential time windows. Performance trends are analyzed to identify patterns of degradation, recovery, and consistency over time. A decline in performance in later windows is interpreted as evidence of concept drift, while stable performance indicates robustness to changing data distributions.

Furthermore, the evaluation framework compares model behavior under different retraining strategies. This enables the study to assess not only the predictive capability of models but also their ability to maintain performance over time. By combining metric-based evaluation with temporal analysis, the study provides a realistic and comprehensive assessment of model performance in dynamic fraud detection environments.

Chapter 4: Data Understanding and Exploratory

Data Analysis

4.1 Data Description

The dataset used for this study is the IEEE-CIS Fraud Detection dataset, which was released as part of a Kaggle competition to improve existing systems for detecting fraud in a live financial environment. The dataset is provided by Vesta Corporation, a leading provider of e-commerce payment solutions. The dataset is a collection of real-world data gathered from online payment systems. The dataset is designed to support the development and evaluation of machine learning models for fraud detection, particularly in large-scale transaction environments where accurate detection and reduction of false positives are critical.

The dataset comprises about 590,000 transaction records, with a total number of 393 anonymized features, making it high-dimensional data suitable for advanced machine learning analysis. The dataset comes in two main parts, which include transaction data consisting of transaction-related information and the identity data containing extra attributes about the user or device making that particular transaction. These two datasets are linked using the primary key feature *TransactionDT*.

The features in the dataset capture a wide range of transactional, Behavioral, and contextual information. These include transaction-level attributes, payment card characteristics, location-related variables, historical behavioral aggregates, and a large set of engineered features designed to capture complex relationships within the data. Due to confidentiality constraints, many of the features are anonymized, particularly those labelled as Vxxx, which represents derived variables created by the provider (Vesta Corporation).

A summary of the key features used in this study and their descriptions are provided in Table 4.1.

Feature	Description
<i>TransactionDT</i>	Represents the time difference relative to predefined reference timestamp
<i>TransactionAMT</i>	Indicates monetary value of the transaction in USD
<i>ProductCD</i>	Product code, identifies the category or type of product involved in the transaction
<i>card1 – card6</i>	Payment card information, contains card-related attributes such as category, issuing institution, and geographic information
<i>addr</i>	Billing location details, billing country (<i>addr2</i>) and billing region (<i>addr1</i>)
<i>dist</i>	Distance
<i>P_emaildomain/</i> <i>R_emaildomain</i>	Correspond to the email domains of the purchaser and recipient
<i>C1 - C14</i>	Counting, such as how many addresses are found to be associated with the payment card, etc. The actual meaning is masked.
<i>D1 – D15</i>	Timedelta, such as days between previous transactions, etc
<i>M1 – M9</i>	Match, such as names on card and addresses, etc
<i>Vxxx</i>	Engineered features generated by Vesta, representing complex relationships such as ranking and interaction patterns

Table 4. 1: Feature Descriptions

A critical aspect of the dataset is the presence of the *TransactioDT* feature, which represents the time elapsed since a reference point. Though there is no information on the exact date, this

variable allows us to order the transactions chronologically, which is crucial for any temporal analysis. It also makes this dataset an excellent choice for exploring the concept of concept drift.

Overall, the IEEE-CIS dataset provides a realistic and complex environment for analyzing fraud detection systems. Its combination of high dimensionality, temporal structure, and imbalanced class distribution makes it well-suited for investigating the temporal stability of the machine learning models under concept drift conditions, which is the primary focus of this study.

4.2 Exploratory Data Analysis

4.2.1 Introduction

Exploratory Data Analysis (EDA) plays a critical role in understanding the structural and statistical properties of the dataset prior to model development. In the context of fraud detection under concept drift, EDA is not limited to descriptive statistics but is used as a tool to investigate temporal dynamics, data instability, and evolving feature behaviour. This section presents a structured analysis of the dataset, focusing on class imbalance, temporal ordering, transaction dynamics, feature stability, and drift quantification using PSI and CSI. These analyses provide empirical justification for the temporal evaluation framework and retraining strategies employed in this study.

4.2.2 Class Imbalance Analysis

The distribution of fraudulent and non-fraudulent transactions was first examined to assess the level of class imbalance. The results indicate that the dataset is highly imbalanced, with the majority of transactions belonging to non-fraud class.

The distribution can be summarized as follows:

- Non-fraudulent transactions (Class 0) – 569,877 instances (90.50%)
- Fraudulent transactions (Class 1) – 20,663 instances (3.50%)

This imbalance is illustrated in Figure 4.1, which shows a clear difference between the two classes.

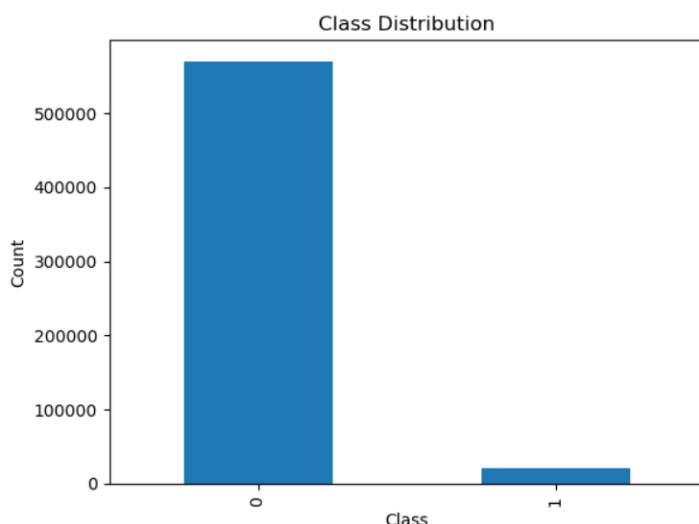


Figure 4. 1: Distribution of fraud (1) and non-fraud (0) transactions

As can be seen from Figure 4.1, fraud cases consist only a minor part of the dataset. This can be explained by the fact that, in real-life cases of detecting fraud, their number will always be much lower compared to normal transactions.

However, this imbalance creates a challenge for ML models. It can cause a tendency of models to favor the majority class. For example, a model that labels every transaction as non-fraudulent would still achieve a high accuracy, even though it completely fails to detect fraud. This shows that accuracy alone is not reliable in this context.

To overcome this issue, this study uses Precision-Recall Area Under Curve (PR-AUC) as the main evaluation metric. PR-AUC is more suitable for imbalanced datasets because it focuses on how well the model identifies the minority class. In addition, ROC-AUC is also used to provide a general view of model performance.

The model training strategy was also impacted by the dataset's imbalance. To make sure the model is exposed to enough fraud transactions, a controlled sampling technique was used during model training. At the same time, the original distribution was kept unchanged in the test data to maintain realistic evaluation conditions.

Overall, all findings prove that class imbalance is one of the key characteristics of the dataset. Taking measures to address this challenge is vital to evaluate the developed models in a relevant manner.

4.2.3 Temporal Ordering and Realistic Data Representation

The dataset was ordered chronologically using *TransactionDT* feature to preserve the natural sequence of transactions. This step ensures that the data reflects the real-world flow of financial transactions, where events occur over time rather than in a random order.

A snippet of the sorted data is provided in Figure 4.2. As evident from the figure, the *TransactionDT* variable values have an increasing trend, which confirms that the dataset has been successfully arranged in chronological order.

	TransactionDT	isFraud
0	86400	0
1	86401	0
2	86469	0
3	86499	0
4	86506	0

Figure 4. 2: Sample of chronologically ordered transactions

Preserving this temporal order is crucial for the study since its goal is to examine how the performance of the models changes over time. Shuffling the data randomly will break the connections that can reveal the impact of concept drift on the prediction accuracy and performance of the models.

In the real process of fraud detection, models are trained using historical data and then used to predict future transactions. As a result, maintaining the chronological order enables the experimental setup to more accurately represent actual deployment circumstances. Additionally, it guarantees that modes are always assessed using the data that occurs after the training phase, lowering the possibility of information leakage.

In summary, temporal ordering provides a realistic foundation for the analysis carried out in this study and supports the investigation of performance stability in a dynamic environment.

4.2.4 Transaction Volume Across Time Windows

To examine how transaction activity varies over time, the number of transactions within each time window was analyzed. Figure 4.3 presents the volume of transactions throughout all time windows.

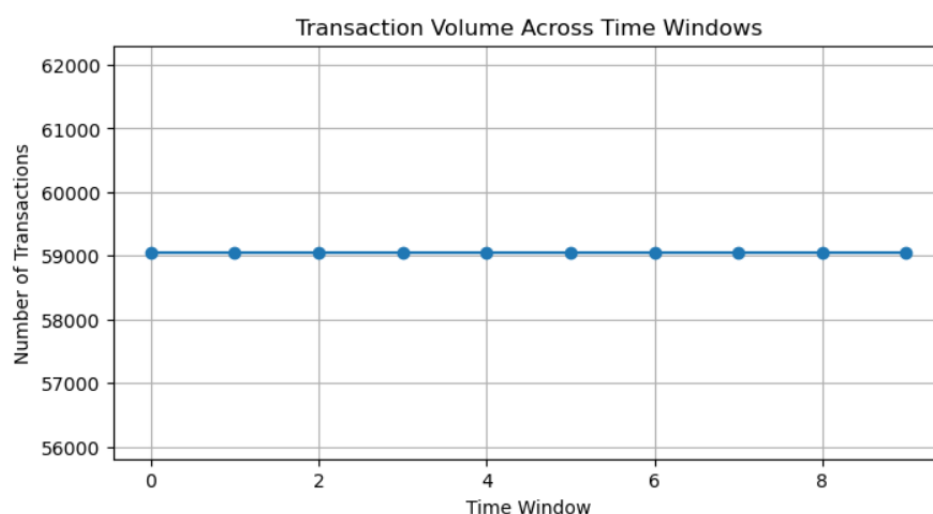


Figure 4.3: Transaction volume across time windows

As shown in Figure 4.3, the number of transactions is constant throughout all the time windows, where each of them includes about 59,054 observations. In addition, the distribution looks quite uniform, with no significant variation in transaction volume.

This uniformity is due to the way the dataset has been split, where each time window has an identical amount of transactions. As a result, the analysis is not influenced by variations in sample size over time.

For this study, maintaining a consistent transaction distribution is crucial since it ensures that comparisons of model performance across time are fair and unbiased. If transaction volume varied significantly between windows, changes in performance could be caused by differences in data quantity rather than changes in underlying patterns.

Although the transaction volume remains stable, this does not mean that dataset itself is stationary. The characteristics of the data, including fraud behavior and feature distribution, may still change across time windows. Therefore, further analysis is required to investigate whether concept drift is present.

4.2.5 Fraud Rate Variation and Behavioral Shifts

To investigate how fraud behavior changes over time, the median transaction amount was analyzed separately for fraudulent and non-fraudulent transactions across time windows. The results are presented in Figure 4.4, with summary of the values shown in Figure 4.5.

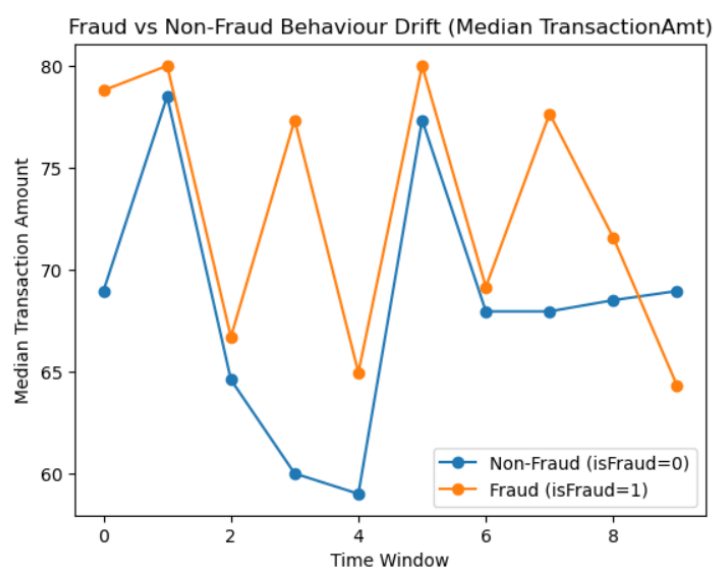


Figure 4.4: Median transaction amount for fraudulent and non-fraudulent transactions across time windows

isFraud	0	1
time_bin		
0	68.950	78.788
1	78.500	80.000
2	64.596	66.696
3	60.000	77.338
4	59.000	64.950
5	77.338	80.000
6	67.950	69.151
7	67.950	77.649
8	68.500	71.563
9	68.950	64.309

Figure 4.5: Median transaction amount across time windows

The median transaction amount for both fraudulent and non-fraudulent transactions changes over time, as Figure 4.4 shows. This suggests that transaction behavior varies over time.

Specifically, in most windows, fraud transactions show larger median values than non-fraud transactions, indicating that transaction quantity is a factor in differentiating the two classes.

Nevertheless, the gap between the fraudulent and non-fraudulent transaction amounts does not remain constant across all time windows. For instance, in time window 0 and 1, it is clear that two groups are distinct from one another, with fraud transactions having significantly larger median values. On the other hand, the difference between the two classes narrow considerably in later windows, and in some cases, the relationship appears to reverse.

This variation in separation is important because it shows that the relationship between the feature (*TransactionAmt*) and target variable (*isFraud*) is not stable over time. In other words, patterns that were useful in identifying fraud transactions during earlier time windows might not be as useful during later ones.

These results offer strong proof of behavioral drift, in which the traits of fraudulent transactions evolve over time. If not addressed, this type of drift can lower model performance by influencing how features contribute to the model predictions.

Additionally, the changes observed across time windows suggest that models trained on past data may find it difficult to generalize to future data, as the underlying patterns of fraud change. This emphasizes how adaptive methods, including retraining techniques are necessary to maintain model effectiveness.

4.2.6 Drift Quantification using CSI and PSI

To further examine the presence of concept drift, statistical measures were used to quantify changes in both feature distribution and feature – target relationships over time. In this study, Population Stability Index (PSI) and Characteristic Stability Index (CSI) were used to evaluate data drift and feature drift respectively.

PSI measures how the distribution of a feature changes between different time periods, while CSI evaluates how the relationship between a feature and the target variable evolves over time. These metrics provide a more objective assessment of drift compared to visual analysis alone.

4.2.6.1 PSI Analysis (Distribution Drift)

The PSI values were calculated by comparing feature distributions between early and later time windows. The results are presented in Figure 4.6, which shows the top features ranked by PSI values.

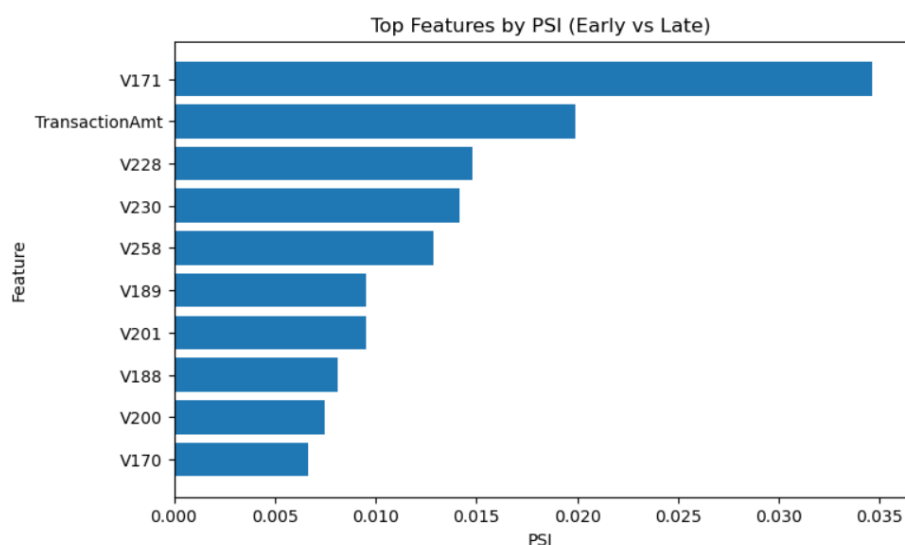


Figure 4.6: Top features ranked by PSI (early vs late comparison)

As shown in Figure 4.6, all PSI values remain below 0.1, with the highest value observed for feature *V171* (one of the Vesta engineered rich feature) at approximately 0.0346, followed by *TransactionAmt* at around 0.0199. According to standard PSI interpretation thresholds, values below 0.1 indicate minor distributional drift.

This implies that the overall data distribution is comparatively stable, despite minor changes in feature distributions over time. The highest levels of change are shown by features like *V171*, *TransactionAmt*, and *V228*, suggesting that these variables may be more prone to temporal variation than others.

Nonetheless, the lack of high PSI values (for example, values above 0.1) suggest that the dataset does not include substantial evidence of distribution drift. Rather than sudden changes, the results point to gradual and moderate modifications.

4.2.6.2 CSI Analysis (Feature Behaviour Drift)

In addition to distributional changes, CSI was used to evaluate whether the relationship between features and the fraud label changes over time. The top features based on CSI values are summarized in Figure 4.7.

	feature	CSI
20	V171	0.048987
17	V230	0.031254
11	V228	0.031062
15	V170	0.020963
9	V258	0.015488
21	V40	0.013830
24	V39	0.013723
26	V43	0.013535
28	V42	0.013468
0	TransactionAmt	0.012600
8	V188	0.011780
6	V200	0.011757
7	V189	0.011029
5	V201	0.011012
27	V79	0.005914
19	V51	0.004281
16	V52	0.002203
14	V87	0.000450
13	V86	0.000398

Figure 4.7: Top features ranked by CSI

The findings demonstrate that all CSI values are comparatively low, with *V171* having the highest values at roughly 0.049, followed by *V230* and *V228* at roughly 0.03. These numbers show that, over time the predictive relationship between characteristics and the target variable is almost constant.

Although some variation is observed, particularly for features such as *V171* and *V230*, the magnitude of change is limited. This suggests that while certain features exhibit minor behavioural changes, there is no strong evidence of significant feature – target relationship drift.

4.2.7 Integrated Evidence of Concept Drift

It is evident that there is a sign of concept drift within the dataset when all these analysis results are considered together since the transaction volumes stay constant throughout time windows (Figure 4.3). However, fraudulent transactions' behaviors differ depending on time periods

according to Figure 4.4. There is no stability in the gap between fraudulent transactions and regular ones, implying that there is an evolution in the correlation between the target and features.

While the CSI results (Figure 4.7) show minor differences in the relationship between features and fraud, the PSI results (Figure 4.6) further demonstrate that feature distributions vary little over time. Despite their lack of significance, these changes are persistent throughout the analysis.

Overall, the dataset exhibits progressive changes over time rather than abrupt or dramatic drift. This shows that the data is not totally stable and confirms the need to investigate the model's performance in certain situations.

4.2.8 Implications for Model Development

The EDA's conclusions have a direct impact on the models' development and evaluation.

PR-AUC is employed as the primary statistic since the class imbalance emphasizes the requirement for suitable evaluation metrics. When models are trained on historical data and tested on future data, the temporal ordering guarantees that the evaluation setting mirrors real-world circumstances.

Even though the transaction volume is consistent, the changes in fraud behaviour and feature relationships suggest that models may not perform equally well over time. The PSI and CSI results confirm that these changes, although small, are present.

This implies that if data changes, a model that was trained on past data could not continue to be useful. Consequently, it's critical to assess if retraining techniques can support long-term performance maintenance.

4.2.9 Summary of Findings

The EDA shows that the dataset has several important characteristics.

First, there is a significant imbalance in the data, which influences how models should be assessed. Second, realistic performance analysis is made possible by the dataset's distinct

temporal structure. Third, fraud behavior and feature patterns vary throughout time frames even though transaction volume remains constant.

These slight but constant changes are confirmed by the PSI and CSI results, suggesting the existence of gradual concept drift.

In conclusion, the dataset is not entirely stationary, and model performance may be impacted by small changes over time. This validates the study's focus on performance stability analysis and the necessity of retraining techniques.

Chapter 5: Implementation

5.1 Introduction

This chapter explains how the proposed temporal fraud detection framework was implemented. Unlike the methodology chapter, which described the overall research design, this chapter focuses on the practical execution of the system and the reasoning behind the main implementation choices.

The implementation was developed in Python and organised as a structured workflow that includes temporal data construction, training data reduction, preprocessing, model training, temporal evaluation, and retraining. The main aim of the implementation was not simply to train accurate classifiers, but to create an experimental setting where model performance could be tracked over time in a realistic fraud detection environment.

A central principle throughout the implementation was to preserve the temporal nature of the data. Since the purpose of this research is to study performance stability under concept drift, the system was designed so that models always learn from past data and are evaluated on future data. At the same time, practical issues such as severe class imbalance and computational cost had to be managed carefully so that the experiments remained realistic and executable.

5.2 Temporal Data Construction

Following the temporal structure established in the EDA, the dataset was used in its chronological form based on the *TransactionDT* feature. This ensured that the natural sequence of transactions was preserved for the implementation of the temporal evaluation framework.

Initially, the intention was to analyse model performance over fixed time periods, such as monthly intervals, to make the experiment closer to reality. However, the *TransactionDT* feature does not contain actual calendar dates or timestamps, but rather a relative time index. As a result, it was not possible to directly segment the dataset into meaningful calendar-based intervals.

The dataset was split into ten consecutive windows, each with roughly 59,000 transactions, in order to overcome this restriction. By treating these windows as proxy time periods, the research was able to replicate the evolution of model performance throughout time.

Equal-sized windows were deliberately chosen instead of fixed-duration intervals so that the comparisons would not be affected by different volumes of data being compared. This is important since this research focuses on how performance deteriorates and how effective the process of retraining becomes. In other words, since the objective is to evaluate the impact of changing data patterns, the sample size needs to remain constant throughout.

In addition, structuring the data in this way allows retraining strategies to be interpreted in a time-based manner. For example, retraining after a fixed number of windows can be conceptually understood as retraining after a certain period (for example, every ‘n’ time intervals), even though exact calendar time is not available in the dataset.

5.3 Training Data Reduction and Class Imbalance Handling

A major practical issue in implementation was the size and imbalance of the dataset. The full merged IEEE-CIS dataset contains around 590,000 transactions, while the fraud class accounts for only about 3.5% of the records. Training directly on the full dataset across repeated temporal experiments would significantly increase computational cost and would also cause the model to be dominated by the majority (non-fraud) class.

To address this, a controlled training data reduction strategy was implemented. This was achieved using a sampling function that:

- ✓ Retains all available fraud samples
- ✓ Samples non-fraud transactions based on fixed ratio
- ✓ Applies a maximum size constraint to the training dataset

The main configuration used in the final implementation was:

- 10 time windows
- Training begins with window 1-6 and then test on later windows
- Maximum training size of 500,000
- Negative-to-positive ratio of 30:1

5.3.1 Sampling Strategy Explanation

The sampling process was designed to maintain a controlled balance between fraud and non-fraud transactions while preserving the importance of the minority class.

Let:

N_{fraud} = number of fraud samples

N_{nonfraud} = number of non-fraud samples selected

r = sampling ratio

Then the sampling ratio is defined as:

$$N_{\text{nonfraud}} = r \times N_{\text{fraud}}$$

Where $r = 30$ in this implementation.

This means that up to thirty non-fraudulent transactions are chosen for each fraudulent transaction that is part of the training data. For example, if a given training window contains 1,000 fraud samples, then up to 30,000 non-fraud samples are included.

The dataset is further reduced proportionately while keeping the same ratio if the total number of chosen samples surpasses the maximum training size of 500,000.

5.3.2 Reasoning Behind the Design

This strategy was chosen to balance two key requirements.

First, all fraud samples were retained because fraud detection is the primary objective of the study. Reducing fraud samples would directly weaken the model's ability to learn meaningful patterns related to fraudulent behaviour.

Second, non-fraud samples were limited to prevent the dataset from becoming excessively imbalanced. Without this control, the model could learn to favour the majority class, leading to poor fraud detection performance despite high overall accuracy.

The selected ratio of 30:1 was chosen as a compromise between class representation and computational feasibility. A smaller ratio would increase class balance but risk losing useful non-fraud patterns, while a larger ratio would increase imbalance and reduce the model's sensitivity to fraud.

Moreover, experiments showed that going above this ratio caused substantial increase in training time and memory requirements, which made the process of conducting numerous temporal trainings unrealistic. In order to preserve meaningful representation of both classes, the 30:1 ratio is the maximum possible setting that could be continuously applied within the computational resources at hand.

Furthermore, restricting the overall training size guarantees that the implementation will continue to be computationally practical throughout numerous experiments with various models and retraining techniques.

5.3.3 Consistency with Research Objective

Additionally, this sampling approach supports the study's main objective. Since the research focuses on performance stability over time, the training process must remain consistent across all time windows and retraining experiments. By using a controlled sampling strategy, it is ensured that variations in model performance are caused by shifts in data patterns rather than irregularities in the amount of training data.

It is crucial to remember that this modification was limited to the training set. To retain realism in performance evaluation, the test windows were preserved in their original format to guarantee that the assessment accurately reflects the dataset's genuine class distribution.

5.4 Preprocessing Pipeline

After completing the data preparation phase, the subsequent process involved data pre-processing. During this phase, it was crucial to guarantee consistency, reproducibility, and no data leakage, particularly since the experiments were repeated in multiple time windows and re-training approaches.

In order to accomplish this, the processing of numerical and categorical features were done independently with respect to each other following a column-wise approach to pre-processing.

The missing values in the case of numerical features were dealt with through the process of imputation. The one hot encoding was performed on categorical features to make them numerical for training purposes.

The reason why this is significant is because applying transformations using the test dataset would result in leakage, leading to inaccurate performance results.

A unified preprocessing pipeline was used for all the models and experiments conducted. It was essential in making sure that consistent transformations would be applied to all models, regardless of the type of retraining being done.

the study includes a number of models and experiments performed repeatedly using various retraining approaches, it makes sense not to preprocess data manually each time but include the preprocessing phase into the model pipeline.

5.5 Model Implementation

Three classification models were implemented in the final system:

- Logistic Regression
- Random Forest
- XGBoost

These models were selected to represent different levels of complexity and different responses to temporal change. Rather than choosing models with similar behaviour, the intention was to compare a simple linear model, a robust tree-based ensemble model, and a high-performance boosting model within the same temporal evaluation framework.

5.5.1 Logistic Regression

Logistic Regression was selected as the baseline classifier. The reasons for using Logistic Regression were its simplicity, popularity, and high interpretability. In the current implementation of the model, the following configuration settings were used:

- ❖ Max_iter = 300
- ❖ Solver = “saga”
- ❖ Class_weight = “balanced”
- ❖ Random_state = 42

The use of class_weight = “balanced” was important because of the strong class imbalance in the dataset. This setting automatically gives greater importance to the minority fraud class during training. The saga solver was used because it is efficient for large-scale datasets and works well with the chosen setup.

This model serves as a useful reference point. Since it learns a linear decision boundary, it allows the study to observe how a simpler model behaves when fraud patterns shift over time.

5.5.2 Random Forest

Random Forest was implemented as a more robust non-linear model. It was chosen because fraud detection data often contains complex feature interactions that cannot be captured well by a linear classifier.

In the implementation, Random Forest was configured with:

- ❖ Class_weight = “balanced_subsample”
- ❖ N_jobs = -1
- ❖ Random_state = 42

The balanced_subsample option was used so that class weighting is applied within each bootstrap sample, which helps the model remain more sensitive to fraud cases without requiring full oversampling. The n_jobs = -1 setting allows all available processor cores to be used, which is useful because repeated temporal training and retraining can be computationally expensive.

Random Forest was expected to provide more stable performance than Logistic Regression because it can model non-linear patterns while also being less sensitive to noise than a single decision tree.

5.5.3 XGBoost

XGBoost was implemented as the high-performance model in the study. It was included because boosting methods are well known for strong performance on structured tabular datasets, especially in fraud detection tasks.

In the implementation, XGBoost was configured with:

- ❖ Objective = “binary:logistic”
- ❖ Eval_metric = “aucpr”
- ❖ Random_state = 42
- ❖ N_jobs = -1
- ❖ Scale_pos_weight = $N_{\text{nonfraud}} / N_{\text{fraud}}$

The scale_pos_weight parameter was particularly important. It was calculated dynamically from the sampled training data so that the model places more emphasis on the minority class. This was necessary because even after training data reduction, fraud cases still remain far fewer than non-fraud cases.

XGBoost was expected to achieve stronger predictive performance, but it was also expected to be more sensitive to temporal changes in the data. This made it especially useful for studying the trade-off between performance and stability under concept drift.

5.5.4 Unified Model Pipeline

The identical preprocessing pipeline was used to implement each of the three models. This indicates that model fitting, feature transformation, and preprocessing were all carried out in a single, structured procedure.

For two reasons, this was significant. In order to make the comparison fair, it first made sure that every model was assessed using the identical preprocessing conditions. Secondly, it decreased the possibility of inconsistent results between numerous windows and retraining techniques in repeated studies.

The final pipeline for each model followed this structure:

Training data → Preprocessing → Model Fitting → Prediction Scores

This unified pipeline was repeated consistently for the no-retraining, fixed-retraining, and performance-triggered retraining experiments.

5.5.5 Temporal Hyperparameter Tuning

In addition to implementing the models, a temporal hyperparameter tuning stage was also included. Instead of using random cross-validation, the tuning process was aligned with the temporal structure of the study.

The tuning process was carried out as follows:

- Early time windows were used as the tuning training set
- The following window was used as the validation set
- Each parameter combination was evaluated using PR-AUC
- The parameter set with the highest PR-AUC was selected

This approach was used because random tuning would break the temporal structure of the project and could lead to unrealistic parameter selection. By tuning in a time-aware way, the selected model settings remained consistent with the overall research design.

Table 5.1 lists the parameter grids used in the finished implementation:

Model	Parameter	Values
Logistic Regression	C	{0.1, 1, 5}
Random Forest	n_estimators	{100, 200}
	max_depth	{8, 12}
	min_samples_split	{10, 20}
	min_sample_leaf	{5, 10}
XGBoost	n_estimators	{100, 200}
	max_depth	{4, 6}
	learning_rate	{0.05, 0.1}
	subsample	{0.8}
	Colsample_bytree	{0.8}

Table 5.1: Hyperparameter grid for models

Optimal hyperparameter settings determined after temporal tuning have been presented below. For Logistic Regression, $C = 0.1$ is the optimal value, implying the need for strong regularization since it helped improve generalization ability due to class imbalance. The optimal Random Forest configuration was found to consist of 200 estimators, maximum depth

of 12, and minimum samples for splitting and leaves of 10. XGBoost had the highest performance in terms of the chosen performance metric. Optimal hyperparameter settings include 200 estimators, maximum depth of 6, learning rate of 0.1, and subsampling fractions of 0.8. These settings resulted in the highest PR-AUC values from the search space defined and were utilized throughout the experiment.

This tuning stage improved the reliability of the models while still keeping the implementation aligned with the temporal nature of the research.

5.5.6 Implementation Rationale

In general, the model implementation strategy was tailored in accordance with the overall objective of the study to evaluate the dynamics of various fraud detection model behavior under concept drift. Thus, Logistic Regression model is utilized as a straightforward approach to serve as a benchmark model whose behavior can be easily observed during the analysis. In addition, the inclusion of Random Forest is intended to address the needs for a non-linear approach that would provide better stability, which could contribute to its capability of dealing with more complex data. XGBoost, on the other hand, is recognized as a highly accurate predictor with relatively low stability, making it prone to changes in concept drift. The use of the above models allows for the evaluation not only of predictive accuracy but also temporal robustness.

5.6 Temporal Evaluation Logic

The temporal evaluation stage is the core of the implementation. In the final system, the first 6 windows are used as the initial training base, and testing begins from the subsequent window. This means that each model is first trained on the earlier part of the timeline and then evaluated sequentially on later windows.

This setup was selected to provide a sufficiently large historical training base before performance evaluation begins. Using too few training windows could result in poor model performance due to insufficient learning data rather than the presence of concept drift. By starting with six windows, this risk is reduced, and a more stable initial model is established.

This configuration also ensures that enough future windows remain available to observe performance degradation and potential recovery under retraining. Since the study focuses on performance stability, a single evaluation point would not be sufficient. Therefore, performance is recorded repeatedly across multiple future windows to capture temporal trends.

5.7 Retraining strategy Implementation

The retraining component is central to this study, as it directly addresses how models adapt to changing data patterns.

5.7.1 No-Retraining Baseline

The first implemented strategy is a no-retraining baseline. In this setup, each model is trained once on the initial training windows and then evaluated on all subsequent windows without any updates.

By using this approach as a reference condition, performance decline can be seen in its most pure form. Without such a baseline, it would be difficult to assess whether retraining strategies provide meaningful improvements.

5.7.2 Fixed Retraining

The second strategy is fixed retraining, where models are retrained at regular intervals. In the implementation, retraining was performed every:

- 1 window
- 2 windows
- 3 windows

A more thorough examination of the behavior of the model is made possible by the use of several retraining frequencies. While retraining every three windows is more effective but takes longer to respond to changes, retraining every one window is a very responsive but computationally costly method. A middle ground between these two extremes is achieved by retraining every two windows.

This configuration makes it possible to assess the trade-off between computational cost and adaption speed.

5.7.3 Performance – Triggered Retraining

The third strategy is performance-triggered retraining, where retraining is initiated only when a significant drop in performance is observed.

PR-AUC is the main trigger metric used in the implementation to track performance over time. It defines relative performance drop thresholds of 5%, 10%, and 15% to identify varying levels of degradation in model effectiveness. To ensure that retraining is not started because of temporary fluctuations, a persistence condition is also added, which requires the performance decline to be shown over two consecutive time periods before initiating a reaction.

This design prevents retraining from being triggered by temporary fluctuations in performance. Requiring the performance drop to persist across multiple windows ensures that retraining is based on consistent degradation rather than noise.

Analyzing the strategy's sensitivity is made possible by testing several thresholds. Since it offers a fair trade-off between responsiveness and stability, the 10% relative decline with a persistence of two windows was chosen as the main setting. While a greater threshold may delay adaption, a lower threshold might cause retraining too frequently.

5.8 Performance Measurement

For evaluation, the implementation records both PR-AUC and ROC-AUC for each model and time window. PR-AUC is given greater importance because the fraud class is rare, and the problem is inherently imbalanced. ROC-AUC is also included to provide an overall measure of classification performance.

In addition to raw performance values, summary measures are computed, including:

- mean performance across windows
- variability across time
- performance degradation from the first to the last test window

These measures are important because the study focuses not only on predictive accuracy, but also on how stable model performance remains over time.

5.9 Summary

Instead of generating a single static classification result, the system was made to match the actual research problem. The system analyzes various model types, maintains temporal order, corrects class imbalance, and assesses various retraining approaches over sequential time periods.

The two main principles of the overall design are control and realism. The implementation maintains consistency and interpretability while reflecting the real-world operation of fraud detection systems. This offers a solid basis for examining how model performance changes with slow concept drift.

Chapter 6: Results and Discussions

6.1 Introduction

This chapter presents and analyses the results obtained from the temporal fraud detection framework. The main objective is to evaluate how model performance changes over time and to examine the effectiveness of different retraining strategies under concept drift.

The analysis is based primarily on PR-AUC, which is more suitable for imbalanced datasets, with ROC-AUC used as a supporting metric. Rather than focusing only on absolute performance values, this chapter emphasizes performance trends, stability, and the impact of retraining.

6.2 Overall Model Performance

The overall performance of the models is summarized in Table 6.1.

Model	Average PR-AUC	Interpretation
Logistic Regression	~ 0.09 – 0.10	Weak (Baseline)
Random Forest	~ 0.45	Strong
XGBoost	~ 0.55	Very Strong

Table 6.1: Average PR-AUC across models

The results demonstrate distinct differences in performance between different models. Logistic Regression obtains low PR-AUC scores consistently, suggesting that a linear decision boundary is not enough to detect fraud in this dataset due to its complexity.

On the other hand, the Random Forest model performs much better, highlighting the need for non-linear relationship modelling. XGBoost demonstrates the best PR-AUC scores, which proves its efficacy in structured fraud detection datasets.

Although XGBoost performs best from a prediction perspective, it does not necessarily prove its reliability over time. The primary concern in this study is whether these models will maintain their performance over time.

6.3 Performance Degradation Over Time

To evaluate the presence of concept drift, model performance was analysed across sequential time windows using the no-retraining strategy. This allows observation of how each model behaves when trained on historical data and applied to future transactions without any updates.

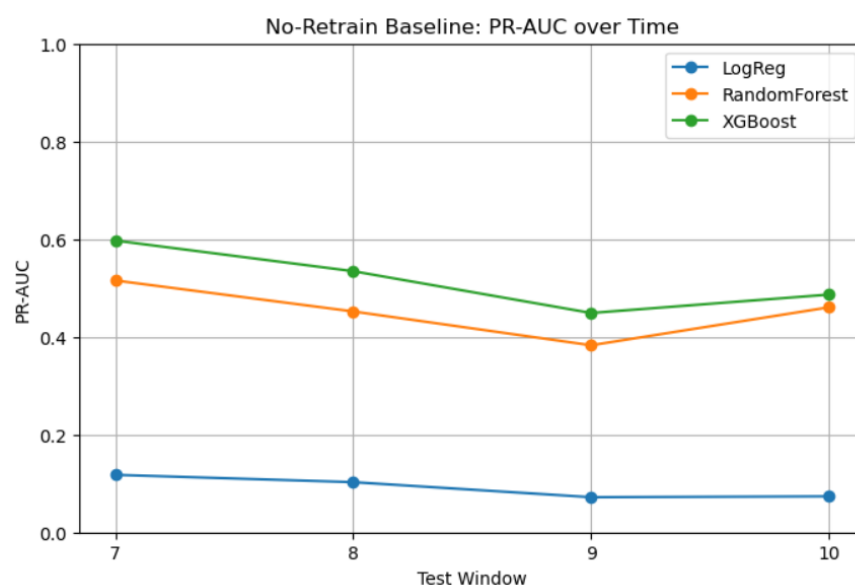


Figure 6.1: PR-AUC vs Time Windows – No retraining for all models

Time Window	Logistic Regression	Random Forest	XGBoost
Window 7	0.1177	0.5154	0.5971
Window 8	0.1028	0.4521	0.5344
Window 9	0.0720	0.3828	0.4489
Window 10	0.0737	0.4606	0.4866

Table 6.2: PR-AUC trend (no retraining)

In Figure 6.1 and Table 6.2, one can clearly observe a trend of performance deterioration in all the three models, even though the extent of the deterioration is different.

The case of the Random Forest classifier is also worth considering. Here, PR-AUC goes down from about 0.5154 in window 7 to 0.3828 in window 9, showing substantial performance degradation. Although a slight recovery is observed in window 10, the model gets worse at detecting fraud when applied to more recent time windows.

Similar observations can be made for XGBoost. Despite performing better than other algorithms, PR-AUC for the algorithm goes down from 0.5971 to 0.4489, meaning that even high-performance models suffer from data shifts. This translates to an overall decline of about 18.5%, the highest of the models.

At the same time, the performance of the Logistic Regression model seems to be relatively steady. The metric varies between 0.1177 to 0.0737. However, as the model is already underfitting the data, this stability results from its poor modeling power rather than tolerance to drift.

Model	Initial PR-AUC	Final PR-AUC	Decrease	Interpretation
Logistic Regression	0.1177	0.0737	Small	Stable but weak
Random Forest	0.5154	0.4606	Moderate	Performance Degradation
XGBoost	0.5971	0.4866	Large	High performance but strongly affected

Table 6.3: Performance Degradation Summary

The performance degradation problem is further emphasized in Table 6.3 above. XGBoost exhibits the greatest decrease in performance, showing significant sensitivity to changes in the pattern of data. On the other hand, the decline in performance for Random Forest is moderate and minor for Logistic Regression because of the latter's low predictive power.

It can be seen from the findings that the drop in performance is consistent regardless of the model used, thereby proving that this is not a model-related problem, but rather a result of data changes.

In summary, concept drift is empirically proven since models perform poorly with time against future transactions.

6.4 Effect of Retraining Strategies

To evaluate the impact of retraining under concept drift, model performance was compared across no-retraining, fixed retraining (every 1, 2, and 3 windows), and performance-triggered retraining.

Model	Strategy	Mean PR-AUC	Degradation (%)
Logistic Regression	No Retraining	0.0916	37.39%
	Fixed (1)	0.1019	29.85%
	Fixed (2)	0.1000	33.88%
	Fixed (3)	0.0993	29.85%
	Performance - Triggered	0.0935	30.83%
Random Forest	No Retraining	0.4527	10.64%
	Fixed (1)	0.4661	5.87%
	Fixed (2)	0.4596	8.63%
	Fixed (3)	0.4593	5.87%
	Performance - Triggered	0.4552	8.68%
XGBoost	No Retraining	0.5167	18.50%
	Fixed (1)	0.5534	4.66%
	Fixed (2)	0.5316	11.03%
	Fixed (3)	0.5275	4.66%
	Performance - Triggered	0.5252	12.82%

Table 6.4: PR-AUC comparison across retraining strategies for all models

It is evident that the retraining process significantly enhances the performance of the machine learning model, outperforming the no-retraining approach.

All retraining techniques for Logistic Regression yield just slight gains. The mean PR-AUC rises marginally from 0.0916 to about 0.10, showing that retraining has little effect because of the model's poor capacity.

Retraining Random Forest produces observable gains. The degradation decreases from 10.64% (no retraining) to 5.87% under fixed retraining, suggesting that regular updates support performance maintenance. Retraining every one and three windows yields comparable improvements among the fixed techniques, although retraining every two windows exhibits marginally more degradation.

Retraining has the biggest effect on XGBoost. Performance declines by 18.50% in the absence of retraining, the largest of the models. However, degradation drops to just 4.66% with fixed retraining every 1 window, indicating a robust recovery. Retraining every two windows leads to more degradation, but retraining every three windows displays similar results.

Additionally, performance-triggered retraining enhances all models' performance. In most instances, though, its means PR-AUC is marginally lower than fixed retraining. Despite this, it is more efficient because it requires fewer retraining procedures to accomplish these benefits.

While ROC-AUC values exhibit less variety than PR-AUC, they are nonetheless generally high across all techniques (usually above 0.85 for tree-based models). This suggests that in unbalanced datasets, ROC-AUC is less sensitive to performance degradation. Consequently, in this study, PR-AUC offers a more significant indicator of retraining effectiveness.

Overall, results support the effectiveness of retraining in lowering performance degradation and enhancing model stability over time.

6.5 Analysis of Retraining Behaviour

While retraining improves performance, the behaviour of different retraining strategies varies depending on how updates are applied and how frequently they occur.

6.5.1 Fixed retraining Behaviour

Fixed retraining updates models at predefined intervals. The results show that retraining frequency plays a critical role in performance stability.

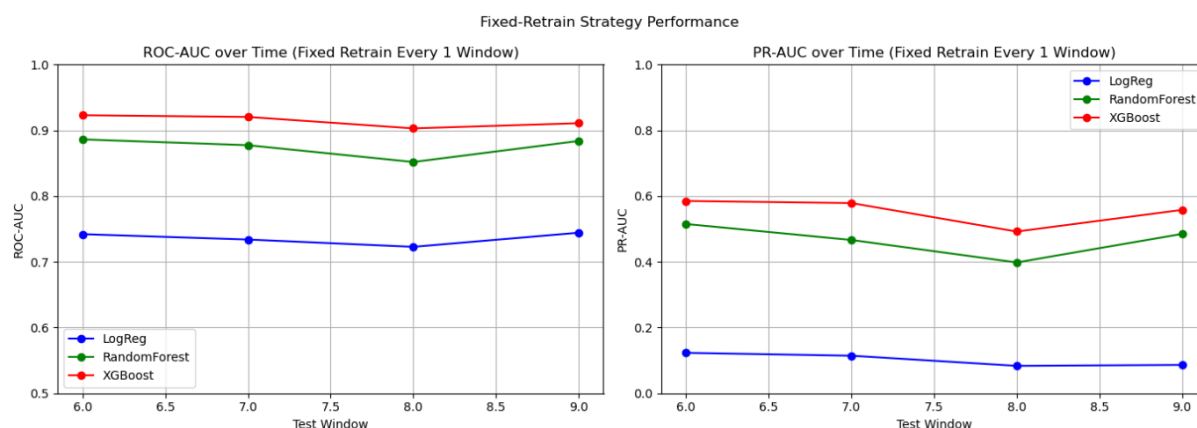


Figure 6.2: Model performance over time under fixed retraining every 1 window (PR-AUC and ROC-AUC)

Figures 6.2 to Figure 6.4 show how fixed retraining techniques behave at various intervals. The models perform the best and most consistently over all time windows when retraining is done after each window. Specifically, XGBoost frequently maintains high PR-AUC values with little degradation (around 4.66%), suggesting that regular updates allow for ongoing adaptation to changing data patterns. Because of its limited learning potential, Logistic Regression only slightly improves in this situation, although Random Forest likewise exhibits steady behavior.

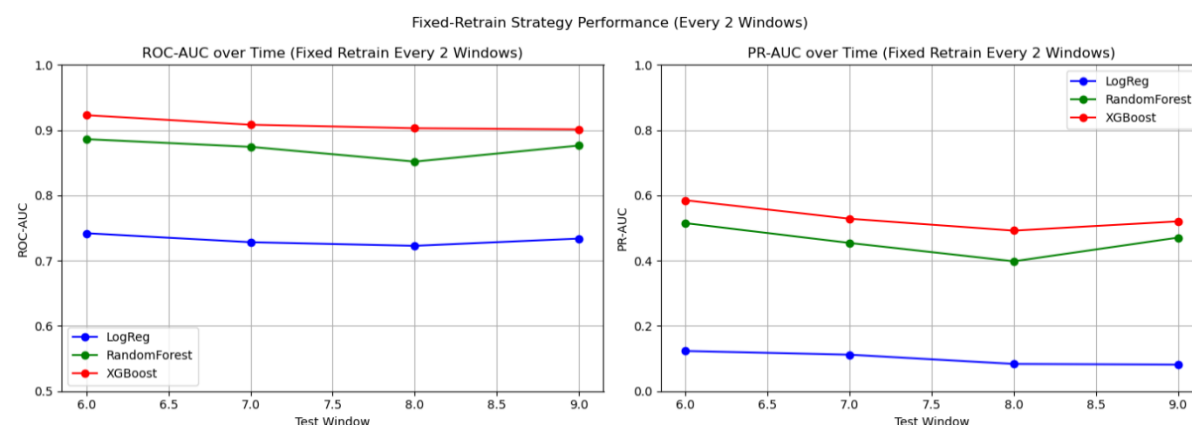


Figure 6.3: Model performance over time under fixed retraining every 2 windows (PR-AUC and ROC-AUC)

Performance is still quite good but varies more noticeably when retraining is done every two windows. The models appear to be temporarily impacted by concept drift prior to being

updated, as evidenced by a slow drop followed by rebound at retraining points. This method offers a fair trade-off between preserving performance and cutting computational expenses.

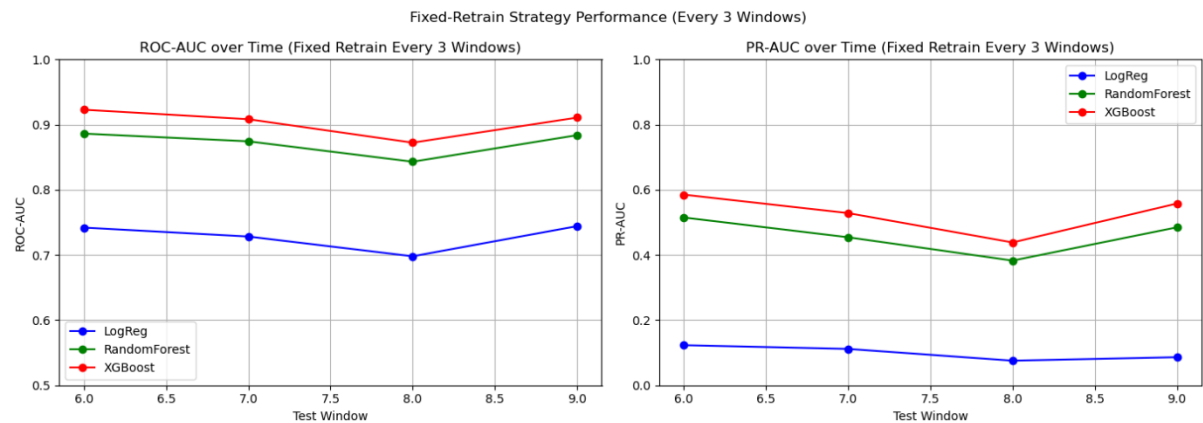


Figure 6.4: Model performance over time under fixed retraining every 3 windows (PR-AUC and ROC-AUC)

Retraining every three windows, on the other hand, causes more noticeable variations in performance. Even if models like XGBoost continue to perform very well, there is a noticeable decline prior to retraining, especially in the vicinity of intermediate windows. This suggests that less frequent updates permit drift to build up, resulting in transient decline prior to recovery. Overall, these findings demonstrate that stability is improved by increasing retraining frequency; nevertheless, when computing efficiency is a top concern, lower frequencies might be favored.

6.5.2 Performance – Triggered Retraining Behaviour

Performance-triggered retraining provides a more adaptive approach by updating models only when a significant performance drop is detected.

In this study, the selected configuration (10% threshold with 2-window persistence) triggered retraining at Window 8 for all models.

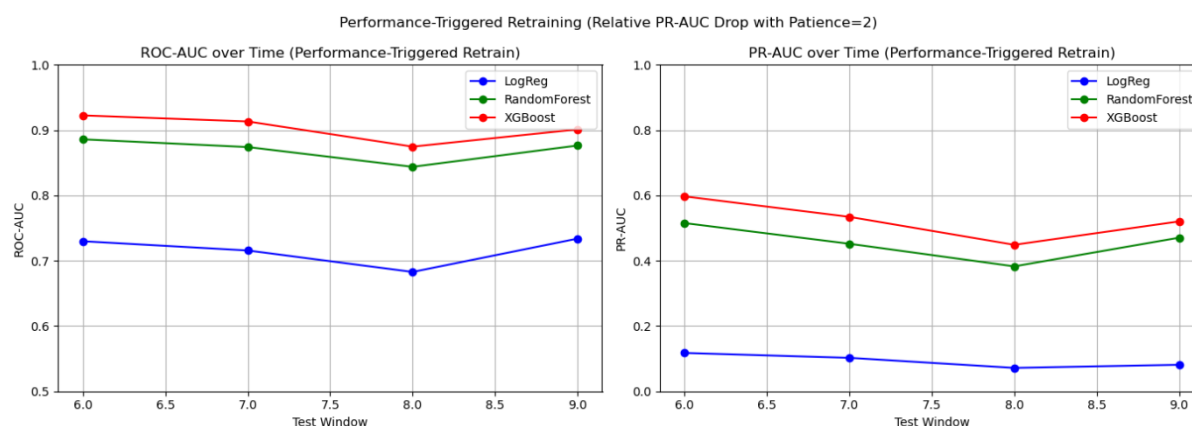


Figure 6.5: Performance of models under performance-triggered retraining (PR-AUC and ROC-AUC over time)

Model	Window	PR-AUC Before	PR-AUC After
Logistic Regression	8	0.0720	0.0814
Random Forest	8	0.3828	0.4707
XGBoost	8	0.4489	0.5205

Table 6.5: Adaptive retraining event

The results from Figure 6.5 and Table 6.5 show that retraining is triggered only when meaningful degradation occurs, leading to immediate recovery in performance.

This strategy successfully avoids unnecessary updates and lowers computational overhead while maintaining acceptable performance levels by putting efficiency before routine. However, the total performance tends to stay somewhat lower than the most aggressive fixed retraining schemes because retraining is only initiated if demonstrable degradation occurs. There is a trade-off with this delay: you save a lot of resources at the expense of the highest level of accuracy that comes from more regular, planned updates.

6.5.3 Identification of Best Strategy

Based on the results, the effectiveness of retraining strategies can be summarised as follows:

- **Best Performance:** Fixed retraining every 1 window (especially for XGBoost)
- **Best balance between performance and efficiency:** Performance-triggered retraining (10% threshold, patience = 2)
- **Moderate option:** Fixed retraining every 2 windows

This suggests that there isn't a single method that works for everyone. Rather, the decision is based on the trade-off between computational cost and performance.

Chapter 7: Limitations and Conclusions

7.1 Limitations of the study

Although this study provides meaningful insights into the temporal stability of fraud detection models under concept drift, several limitations should be acknowledged.

One of the main limitations is related to the dataset structure. Although there is a temporal variable (*TransactionDT*) in the IEEE-CIS dataset, actual calendar timestamps are not included. Consequently, real-world time intervals like days or months could not be used to analyze performance. Rather, the dataset was considered as proxy time periods and split into equal-sized time windows. While this approach preserves temporal order, it limits the ability to interpret results in terms of real-world operational timelines.

Another limitation is the controlled sampling strategy used during training. Although necessary for handling class imbalance and computational constraints, the use of a fixed negative-to-positive ratio (30:1) may influence model learning. While all fraud cases were preserved, some non-fraud information was reduced, which could affect the model's ability to fully capture the distribution of legitimate transactions. In addition, the ratio was partly constrained by available computational resources, meaning larger-scale experiments were not feasible.

The study is also limited in terms of model selection. Only three models were evaluated: Logistic Regression, Random Forest, and XGBoost. While these models were deliberately chosen to represent different levels of complexity and behaviour, other advanced approaches such as deep learning models or online learning algorithms were not explored. Including such models could provide additional insights into performance stability under concept drift.

Another limitation is related to the scope of retraining strategies. The study focuses on fixed and performance-triggered retraining, but does not include more advanced adaptive techniques such as continuous online learning or ensemble-based drift adaptation. These approaches may offer improved adaptability in highly dynamic environments.

Lastly, models are assessed on predetermined time frames in an offline environment. Real-time decision-making is required in real-world fraud detection systems due to the constant flow of

data. As a result, even though the results offer valuable insights into temporal behavior, more research is needed to confirm these conclusions in a real-world deployment setting.

7.2 Conclusions

Using a temporal assessment framework, this study investigated the behavior of fraud detection algorithms under concept drift across time. The findings validate the existence of concept drift in the dataset by demonstrating that model performance steadily declines when trained on historical data and applied to future transactions.

Random Forest exhibited more steady behavior, while XGBoost had the best prediction performance but was also the most impacted by temporal fluctuations. Because of its poor modeling capabilities, logistic regression remained persistently weak.

The findings also demonstrate that retraining plays a key role in maintaining performance. Fixed retraining, especially at shorter intervals, provides the best performance, while performance-triggered retraining offers a more efficient alternative with fewer updates.

Overall, this study shows that fraud detection tasks require more than just static evaluation. Retraining techniques are crucial to ensuring long-term model reliability in dynamic contexts, and a temporal approach is required to comprehend model behavior realistically.

References

- Arner, D. W., Barberis, J., & Buckley, R. P. (2016). *The evolution of FinTech: A new post-crisis paradigm?* *Georgetown Journal of International Law*, 47(4), 1271–1319.
- Baesens, B., Van Vlasselaer, V., & Verbeke, W. (2021). *Fraud analytics using descriptive, predictive, and social network techniques*. Wiley.
- Bahnsen, A. C., Aouada, D., & Ottersten, B. (2014, December). Example-dependent cost-sensitive logistic regression for credit scoring. In *2014 13th International conference on machine learning and applications* (pp. 263-269). IEEE.
- Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3), 602–613.
- Bifet, A., & Gavalda, R. (2007, April). Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM international conference on data mining* (pp. 443-448). Society for Industrial and Applied Mathematics.
- Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16, 321-357.
- Chen, T., & Guestrin, C. (2016, August). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785-794).
- Chen, Y., Zhao, C., Xu, Y., Nie, C., & Zhang, Y. (2025). Deep learning in financial fraud detection: Innovations, challenges, and applications. *Data Science and Management*.
- Compagnino, A. A., Maruccia, Y., Cavuoti, S., Riccio, G., Tutone, A., Crupi, R., & Pagliaro, A. (2025). An introduction to machine learning methods for fraud detection. *Applied Sciences*, 15(21), 11787.
- Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). Credit card fraud detection: A realistic modeling approach. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3784–3797.

- Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. *IEEE Symposium Series on Computational Intelligence*, 159–166.
- Dal Pozzolo, A., Caelen, O., Le Borgne, Y. A., Waterschoot, S., & Bontempi, G. (2014). Learned lessons in credit card fraud detection from a practitioner perspective. *Expert systems with applications*, 41(10), 4915-4928.
- Davis, J., & Goadrich, M. (2006, June). The relationship between Precision-Recall and ROC curves. In *Proceedings of the 23rd international conference on Machine learning* (pp. 233-240).
- Fiore, U., De Santis, A., Perla, F., Zanetti, P., & Palmieri, F. (2019). Using generative adversarial networks for improving classification effectiveness in credit card fraud detection. *Information Sciences*, 479, 448-455.
- Gama, J. (2010). *Knowledge discovery from data streams*. CRC Press.
- Gama, J., Medas, P., Castillo, G., & Rodrigues, P. (2004, September). Learning with drift detection. In *Brazilian symposium on artificial intelligence* (pp. 286-295). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 1–37.
- Jurgovsky, J., Granitzer, M., Ziegler, K., Calabretto, S., Portier, P. E., He-Guelton, L., & Caelen, O. (2018). Sequence classification for credit-card fraud detection. *Expert systems with applications*, 100, 234-245.
- Krawczyk, B., Minku, L. L., Gama, J., Stefanowski, J., & Woźniak, M. (2017). Ensemble learning for data stream analysis: A survey. *Information Fusion*, 37, 132-156.
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363.
- Ngai, E. W. T., Hu, Y., Wong, Y. H., Chen, Y., & Sun, X. (2011). The application of data mining techniques in financial fraud detection. *Decision Support Systems*, 50(3), 559–569.

- Ngai, E. W., Hu, Y., Wong, Y. H., Chen, Y., & Sun, X. (2011). The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision support systems*, 50(3), 559-569.
- Ozili, P. K. (2018). Impact of digital finance on financial inclusion and stability. *Borsa Istanbul Review*, 18(4), 329–340.
- Phua, C., Lee, V., Smith, K., & Gayler, R. (2010). A comprehensive survey of data mining-based fraud detection research. *Artificial Intelligence Review*, 34(1), 1–14.
- Vives, X. (2019). Digital disruption in banking. *Annual Review of Financial Economics*, 11, 243–272.
- Widmer, G., & Kubat, M. (1996). Learning in the presence of concept drift and hidden contexts. *Machine learning*, 23(1), 69-101.